SQL Level 4: GROUP BY & HAVING

Topic-wise notes with examples and explanations

Sample Table Used in All Examples

Table: `employees(emp_id, name, department, job_title, gender, salary)`. Salaries are in rupees.

emp_id	name	department	job_title	gender	salary
1	Arun	IT	Developer	M	85,000
2	Bhavana	IT	Developer	F	90,000
3	Charan	IT	Tester	M	55,000
4	Divya	IT	Tester	F	58,000
5	Eshwar	IT	Developer	M	100,000
6	Farah	IT	Manager	F	120,000
7	Gita	HR	Recruiter	F	30,000
8	Harish	HR	Recruiter	M	32,000
9	Indira	HR	Manager	F	60,000
10	Jai	Sales	Executive	M	30,000
11	Kavya	Sales	Executive	F	34,000
12	Lokesh	Sales	Executive	M	52,000
13	Meera	Sales	Manager	F	70,000
14	Naveen	Finance	Accountant	M	55,000
15	Omkar	Finance	Accountant	M	50,000
16	Padma	Finance	Accountant	F	60,000

Topic 1: GROUP BY

What it is: GROUP BY collects rows that have the same value in one or more columns into a single group. Instead of getting one output row per employee, you get one output row per group. It is almost always used together with an aggregate function (COUNT, SUM, AVG, MIN, MAX).

Syntax

```
SELECT column, AGGREGATE(column)
FROM table
GROUP BY column;
```

How the grouping happens (visual)

Group (department)	Rows that fall in the group	Rows
IT	Arun, Bhavana, Charan, Divya, Eshwar, Farah	6
HR	Gita, Harish, Indira	3
Sales	Jai, Kavya, Lokesh, Meera	4
Finance	Naveen, Omkar, Padma	3

16 employee rows collapse into 4 group rows. The aggregate function is then applied inside each group.

Example 1: How many employees are in each department?

SQL Query

```
SELECT department, COUNT(*) AS total_employees
FROM employees
GROUP BY department;
```

Output

department	total_employees
Finance	3
HR	3
IT	6
Sales	4

Explanation

The database forms one group for each distinct department, then COUNT(*) counts the rows in every group.

Example 2: GROUP BY without an aggregate (distinct values)

SQL Query

```
SELECT department
FROM employees
GROUP BY department;
```

Output

department
Finance
HR
IT
Sales

Explanation

With no aggregate function, GROUP BY simply returns each distinct value once. It behaves like SELECT DISTINCT department.

Common mistake

```
SELECT department, name, COUNT(*)
FROM employees
GROUP BY department;
```

name is neither in GROUP BY nor inside an aggregate. A group has many names, so SQL cannot decide which one to show. Most databases raise an error.

Correct version

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

Rule: every column in SELECT must either be in GROUP BY or be wrapped in an aggregate function.

Topic 2: Grouping One Column

What it is: The simplest form. You pick a single column, and each unique value of that column becomes one group.

Example 1: Group by department

SQL Query

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

Output

department	emp_count
Finance	3
HR	3
IT	6
Sales	4

Explanation

Four unique departments give four groups.

Example 2: Group by job title

SQL Query

```
SELECT job_title, COUNT(*) AS emp_count
FROM employees
GROUP BY job_title;
```

Output

job_title	emp_count
Accountant	3
Developer	3
Executive	3
Manager	3
Recruiter	2
Tester	2

Explanation

Same idea with a different column. 'Manager' belongs to IT, HR and Sales, but all three are counted together in one 'Manager' group because grouping uses only job_title.

Example 3: Group by gender

SQL Query

```
SELECT gender, AVG(salary) AS avg_salary
FROM employees
GROUP BY gender;
```

Output

gender	avg_salary
F	65,250
M	57,375

Explanation

Two unique values (F and M) give two groups. AVG(salary) is then calculated separately for each gender.

Tip: add ORDER BY to sort the groups, for example `ORDER BY emp_count DESC` to show the biggest group first.

Topic 3: Grouping Multiple Columns

What it is: When you list more than one column in GROUP BY, a group is created for every unique **combination** of those columns. The groups are narrower, so you get more output rows.

Syntax

```
SELECT col1, col2, AGGREGATE(col3)
FROM table
GROUP BY col1, col2;
```

Example 1: Count by department and gender

SQL Query

```
SELECT department, gender, COUNT(*) AS emp_count
FROM employees
GROUP BY department, gender;
```

Output

department	gender	emp_count
Finance	F	1
Finance	M	2
HR	F	2
HR	M	1
IT	F	3
IT	M	3
Sales	F	2
Sales	M	2

Explanation

IT + F, IT + M, HR + F, HR + M and so on are each a separate group. IT has 3 women (Bhavana, Divya, Farah) and 3 men (Arun, Charan, Eshwar).

Example 2: Average salary by department and gender

SQL Query

```
SELECT department, gender, AVG(salary) AS avg_salary
FROM employees
GROUP BY department, gender;
```

Output

department	gender	avg_salary
Finance	F	60,000
Finance	M	52,500
HR	F	45,000
HR	M	32,000
IT	F	89,333.33
IT	M	80,000
Sales	F	52,000
Sales	M	41,000

Explanation

Compares the average pay of men and women inside each department. The order of columns in GROUP BY does not change the groups, only the default row order.

Example 3: Count by department and job title

SQL Query

```
SELECT department, job_title, COUNT(*) AS emp_count
FROM employees
GROUP BY department, job_title;
```

Output

department	job_title	emp_count
Finance	Accountant	3
HR	Manager	1
HR	Recruiter	2
IT	Developer	3
IT	Manager	1
IT	Tester	2
Sales	Executive	3
Sales	Manager	1

Explanation

Now 'Manager' is split into IT Manager, HR Manager and Sales Manager because the department is part of the group key.

Remember: one column gives broad groups, multiple columns give finer groups. Every grouping column in SELECT must appear in GROUP BY.

Topic 4: Aggregate Functions with GROUP BY

What it is: Aggregate functions summarise many rows into one value. With GROUP BY, they run once per group instead of once for the whole table.

Function	Purpose
COUNT(*)	Number of rows in the group
COUNT(column)	Number of non-NULL values in that column
SUM(column)	Total of the values
AVG(column)	Average of the values
MIN(column)	Smallest value
MAX(column)	Largest value

Without GROUP BY the same functions return a single row for the whole table, for example `SELECT AVG(salary) FROM employees;`.

Example 1: Several aggregates together

SQL Query

```
SELECT department,
       COUNT(*)    AS emp_count,
       SUM(salary) AS total_salary,
       AVG(salary) AS avg_salary,
       MIN(salary) AS min_salary,
       MAX(salary) AS max_salary
FROM employees
GROUP BY department;
```

Output

department	emp_count	total_salary	avg_salary	min_salary	m
Finance	3	165,000	55,000	50,000	6
HR	3	122,000	40,666.67	30,000	6
IT	6	508,000	84,666.67	55,000	1
Sales	4	186,000	46,500	30,000	7

Explanation

You can use many aggregate functions in one query. Each one is calculated over the same groups. For IT, the total is 5,08,000, the average is 84,666.67, the minimum is 55,000 and the maximum is 1,20,000.

Example 2: Total salary by job title

SQL Query

```
SELECT job_title, SUM(salary) AS total_salary
FROM employees
GROUP BY job_title;
```

Output

job_title	total_salary
Accountant	165,000
Developer	275,000
Executive	116,000
Manager	250,000
Recruiter	62,000
Tester	113,000

Explanation

SUM adds salaries within each job title group. Developers: 85,000 + 90,000 + 1,00,000 = 2,75,000.

Example 3: Highest paid amount per department, sorted

SQL Query

```
SELECT department, MAX(salary) AS max_salary
FROM employees
GROUP BY department
ORDER BY max_salary DESC;
```

Output

department	max_salary
IT	120,000
Sales	70,000
HR	60,000
Finance	60,000

Explanation

MAX picks the top salary in each group. ORDER BY then sorts the finished groups using the alias.

Topic 5: HAVING

What it is: HAVING filters **groups** after GROUP BY and the aggregate calculations are done. Because it runs after aggregation, HAVING can use aggregate functions such as COUNT(*) or AVG(salary).

Syntax

```
SELECT column, AGGREGATE(column)
FROM table
GROUP BY column
HAVING AGGREGATE(column) condition;
```

Example 1: Departments with more than 5 employees

SQL Query

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Output

department	emp_count
IT	6

Explanation

Step 1: groups are formed and counted (IT 6, HR 3, Sales 4, Finance 3). Step 2: HAVING keeps only groups whose count is above 5. Only IT remains.

Example 2: Departments with average salary above Rs. 50,000

SQL Query

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

Output

department	avg_salary
Finance	55,000
IT	84,666.67

Explanation

Averages are HR 40,666.67, Sales 46,500, Finance 55,000, IT 84,666.67. HAVING removes HR and Sales.

Example 3: Departments with total salary above Rs. 5,00,000

SQL Query

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 500000;
```

Output

department	total_salary
IT	508,000

Explanation

Only IT (5,08,000) crosses 5,00,000. Write the number without commas in SQL.

Common mistake

```
SELECT department, COUNT(*)
FROM employees
WHERE COUNT(*) > 5
GROUP BY department;
```

WHERE runs before grouping, so COUNT(*) does not exist yet. This gives an error.

Correct version

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Topic 6: WHERE vs HAVING

Point	WHERE	HAVING
Filters	Individual rows	Groups
Runs	Before GROUP BY	After GROUP BY
Aggregate functions	Not allowed	Allowed
Used without GROUP BY	Yes	Rarely (whole table is treated as one group)
Typical use	salary > 50000, department = 'IT'	COUNT(*) > 5, AVG(salary) > 50000

Order of execution: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Example 1: WHERE only (filters rows)

SQL Query

```
SELECT department, COUNT(*) AS emp_count
FROM employees
WHERE gender = 'F'
GROUP BY department;
```

Output

department	emp_count
Finance	1
HR	2
IT	3
Sales	2

Explanation

WHERE first throws away all male employees. Then the remaining rows are grouped and counted. This answers: how many women are in each department?

Example 2: HAVING only (filters groups)

SQL Query

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) >= 4;
```

Output

department	emp_count
IT	6
Sales	4

Explanation

All employees are grouped first. HAVING then removes departments with fewer than 4 people. This answers: which departments are large?

Example 3: WHERE and HAVING together

SQL Query

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE gender = 'F'
GROUP BY department
HAVING AVG(salary) > 50000;
```

Output

department	avg_salary
Finance	60,000
IT	89,333.33
Sales	52,000

Explanation

Step 1: WHERE keeps only female employees. Step 2: GROUP BY department. Step 3: AVG(salary) is computed for the women in each department. Step 4: HAVING keeps departments where that average is above 50,000. The female averages are Finance 60,000, IT 89,333.33, Sales 52,000 and HR 45,000, so only HR is removed.

Topic 7: Filtering Grouped Data

What it is: A complete pattern that combines everything: filter rows with WHERE, build groups with GROUP BY, filter groups with HAVING, then sort and limit the result.

Full template

```
SELECT   column, AGGREGATE(column)
FROM     table
WHERE    row_condition
GROUP BY column
HAVING   group_condition
ORDER BY column_or_alias
LIMIT    n;
```

Example 1: Departments with more than 3 employees earning above Rs. 50,000

SQL Query

```
SELECT department, COUNT(*) AS high_earners
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 3;
```

Output

department	high_earners
IT	6

Explanation

WHERE removes everyone earning 50,000 or less. The remaining rows are grouped by department and counted. HAVING keeps departments with more than 3 such employees. IT has 6; Sales and Finance have 2 each; HR has 1.

Example 2: Department with the highest average salary

SQL Query

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC
LIMIT 1;
```

Output

department	avg_salary
IT	84,666.67

Explanation

Average per department, sort high to low, keep only the first row. SQL Server uses SELECT TOP 1, and Oracle 12c+ uses FETCH FIRST 1 ROW ONLY.

Example 3: Job titles with at least 3 people, largest first

SQL Query

```
SELECT job_title, COUNT(*) AS emp_count
FROM employees
GROUP BY job_title
HAVING COUNT(*) >= 3
ORDER BY emp_count DESC;
```

Output

job_title	emp_count
Manager	3
Executive	3
Developer	3
Accountant	3

Explanation

Group by job title, keep groups with 3 or more people, then sort. Developer, Manager, Executive and Accountant each have 3.

Example 4: Filter rows and groups, multiple columns

SQL Query

```
SELECT department, gender, AVG(salary) AS avg_salary
FROM employees
WHERE salary >= 40000
GROUP BY department, gender
HAVING AVG(salary) > 60000
ORDER BY avg_salary DESC;
```

Output

department	gender	avg_salary
IT	F	89,333.33
IT	M	80,000
Sales	F	70,000

Explanation

Rows with salary below 40,000 are removed first. The rest are grouped by department and gender. Only combinations whose average is above 60,000 survive, and they are sorted from highest to lowest.

Quick Revision

I want to...	Use
Remove rows before grouping	WHERE
Remove groups based on COUNT/SUM/AVG	HAVING
Group by combinations	GROUP BY col1, col2
Show the top group	ORDER BY aggregate DESC LIMIT 1
Show only distinct values	GROUP BY column (no aggregate)